

[Event dashboard](#) > Running the Web Application

Running the Web Application

The CLI agent works, but a terminal is not the most welcoming interface. In this section we wrap exactly the same four functions in a Streamlit app so you can see each phase of the workflow happen live in the browser.

 Copied!

Create the Web Application

Inside the folder created in the environment setup i.e. `agentic-ai-workshop`, create a new file called `weather_agent_web.py`  and copy this complete code:

```
1  import streamlit as st
2  import boto3
3  import subprocess
4  import json
5  import time
6  from datetime import datetime
7  from PIL import Image
8  import os
9
10 # Page configuration
11 st.set_page_config(
12     page_title="Weather AI Agent",
13     page_icon="🌤️",
14     layout="wide",
15     initial_sidebar_state="expanded"
16 )
17
18 # Custom CSS for better styling
19 st.markdown("""
20 <style>
21     .step-container {
22         border: 2px solid #e0e0e0;
23         border-radius: 10px;
24         padding: 20px;
25         margin: 10px 0;
26         background-color: #f9f9f9;
27     }
```



```

28     .step-header {
29         font-size: 18px;
30         font-weight: bold;
31         color: #1f77b4;
32         margin-bottom: 10px;
33     }
34     .success-box {
35         border-left: 5px solid #28a745;
36         background-color: #d4edda;
37         padding: 10px;
38         margin: 10px 0;
39         color: #000000;
40     }
41     .error-box {
42         border-left: 5px solid #dc3545;
43         background-color: #f8d7da;
44         padding: 10px;
45         margin: 10px 0;
46         color: #000000;
47     }
48     .info-box {
49         border-left: 5px solid #17a2b8;
50         background-color: #d1ecf1;
51         padding: 10px;
52         margin: 10px 0;
53         color: #000000;
54     }
55 </style>
56 """ , unsafe_allow_html=True)
57
58 def call_claude_sonnet(prompt):
59     """
60     Connect to Claude 4.5 Sonnet via Amazon Bedrock
61     """
62     bedrock = boto3.client(
63         service_name='bedrock-runtime',
64         region_name='us-west-2'
65     )
66
67     try:
68         response = bedrock.converse(
69             modelId='us.anthropic.claude-sonnet-4-5-20250929-v1:0',
70             messages=[
71                 {
72                     "role": "user",
73                     "content": [{"text": prompt}]
74                 }
75             ],
76             inferenceConfig={

```


 A green checkmark icon followed by the text "Copied!".

```

77         "maxTokens": 2000,
78         "temperature": 0.7
79     }
80 )
81
82     return True, response['output']['message']['content'][0]['text']
83
84 except Exception as e:
85     return False, f"Error calling Claude: {str(e)}"
86
87 def execute_curl_command(url):
88     """
89     Execute curl command to fetch API data
90     """
91     try:
92         result = subprocess.run(
93             ['curl', '-s', url],
94             capture_output=True,
95             text=True,
96             timeout=30
97         )
98
99         if result.returncode == 0:
100             return True, result.stdout
101         else:
102             return False, f"Curl command failed: {result.stderr}"
103
104     except subprocess.TimeoutExpired:
105         return False, "Request timed out after 30 seconds"
106     except Exception as e:
107         return False, f"Error executing curl: {str(e)}"
108
109 def generate_weather_api_calls(location):
110     """
111     Use Claude to generate NWS API calls
112     """
113     prompt = f"""
114     You are an expert at working with the National Weather Service (NWS) API.
115
116     Your task: Generate the NWS API URL to get weather forecast data for "{location}"
117
118     Instructions:
119     1. First, determine the approximate latitude and longitude coordinates for thi
120     2. Generate the NWS Points API URL: https://api.weather.gov/points/{{lat}},{{l
121
122     For the coordinates, use your knowledge to estimate:
123     - Major cities: Use well-known coordinates
124     - ZIP codes: Estimate based on the area
125     - States: Use approximate center coordinates

```


 A green checkmark icon followed by the text "Copied!".

```

126 - In case a location description is provided instead of a location name, pleas
127
128 Example for Seattle:
129 https://api.weather.gov/points/47.6062,-122.3321
130
131 Example for largest city in USA:
132 Based on your knowledge, you will establish location is New York City
133 https://api.weather.gov/points/40.7128,-74.0060
134
135 Now generate the API call (Points API) for the established location.
136 Return ONLY the complete Points API URL, nothing else.
137 Format: https://api.weather.gov/points/LAT,LON
138 """
139
140 success, response = call_claude_sonnet(prompt)
141
142 if success:
143     api_url = response.strip()
144     if api_url.startswith('https://api.weather.gov/points/'):
145         return True, [api_url]
146     else:
147         return False, f"AI generated invalid URL: {api_url}"
148 else:
149     return False, response
150
151 def get_forecast_url_from_points_response(points_json):
152     """
153     Extract forecast URL from Points API response
154     """
155     try:
156         data = json.loads(points_json)
157         forecast_url = data['properties']['forecast']
158         return True, forecast_url
159     except (json.JSONDecodeError, KeyError) as e:
160         return False, f"Error parsing Points API response: {str(e)}"
161
162 def process_weather_response(raw_json, location):
163     """
164     Use Claude to process NWS API response
165     """
166     prompt = f"""
167     You are a weather information specialist. I have raw National Weather Service
168
169     Raw NWS API Response:
170     {raw_json}
171
172     Please create a weather summary that includes:
173     1. A brief introduction with the location
174     2. Current conditions and today's forecast

```


 A green checkmark icon followed by the text "Copied!".

```

175 3. The next 2-3 days outlook with key details (temperature, precipitation, win
176 4. Any notable weather patterns or alerts
177 5. Format the response to be easy to read and understand
178
179 Make it informative and practical for someone planning their activities. Focus
180 ""
181
182     success, response = call_claude_sonnet(prompt)
183     return success, response
184
185 # Sidebar with information
186 st.sidebar.title("🤖 About This Agent")
187 st.sidebar.markdown("""
188 This AI agent demonstrates **Agentic AI** principles:
189
190 **🧠 Intelligence**: Uses Claude 4.5 Sonnet to understand locations and plan /
191
192 **🔗 Action**: Automatically calls the National Weather Service API
193
194 **📊 Processing**: Converts complex weather data into readable forecasts
195
196 **💬 Response**: Provides helpful, practical weather information
197 """)
198
199 st.sidebar.markdown("---")
200 st.sidebar.markdown("### 🏗️ Architecture")
201 st.sidebar.markdown("""
202 1. **User Input** → Location name
203 2. **AI Planning** → Generate API calls
204 3. **Points API** → Get forecast office
205 4. **Forecast API** → Get weather data
206 5. **AI Processing** → Create summary
207 6. **Display Results** → Show to user
208 """)
209
210 # Main application
211 st.title("☀️ Weather AI Agent")
212 st.markdown("### ⚡ Powered by Claude 4.5 Sonnet on Amazon Bedrock")
213
214 st.markdown("""
215 This intelligent agent helps you get weather forecasts using the National Weat
216 Enter any location below and watch the AI agent work through its reasoning pro
217 """)
218
219 # Initialize session state for results
220 if 'show_results' not in st.session_state:
221     st.session_state.show_results = False
222
223 # Input section

```


 A light blue speech bubble with a white checkmark icon and the text "Copied!".

```

224 st.markdown("---")
225 location = st.text_input(
226     "🔍 Enter a location name or description:",
227     placeholder="e.g., Seattle, 90210, New York City, National park near Homes
228     help="You can enter city names, ZIP codes, state names, or location descri
229 )
230
231 # Create columns for the buttons
232 button_col1, button_col2 = st.columns([2, 1])
233
234 with button_col1:
235     get_forecast = st.button("🚀 Get Weather Forecast", type="primary")
236
237 with button_col2:
238     clear_results = st.button("🗑️ Clear Results", type="secondary")
239
240 # Clear results functionality
241 if clear_results:
242     st.session_state.show_results = False
243     st.success("🗑️ Results cleared! Enter a new location to get a fresh forec
244
245 if get_forecast:
246     st.session_state.show_results = True
247
248 if st.session_state.show_results and get_forecast:
249     if not location:
250         st.error("❌ Please enter a location name or description.")
251     else:
252         # Create columns for better layout
253         col1, col2 = st.columns([2, 1])
254
255         with col1:
256             st.markdown(f"## Weather Analysis for: **{location}**")
257
258             # Step 1: AI Planning
259             with st.container():
260                 st.markdown('<div class="step-container">', unsafe_allow_html=
261                 st.markdown('<div class="step-header">🧠 Step 1: AI Planning I
262
263                 with st.spinner("Claude is analyzing the location and planning
264                     success, api_calls = generate_weather_api_calls(location)
265
266                 if success:
267                     points_url = api_calls[0]
268                     st.markdown('<div class="success-box">✅ Points API URL g
269                     st.code(points_url, language="text")
270                 else:
271                     st.markdown(f'<div class="error-box">❌ Failed to generat
272                     st.stop()

```



```

273
274         st.markdown('</div>', unsafe_allow_html=True)
275
276     # Step 2: Points API Execution
277     with st.container():
278         st.markdown('<div class="step-container">', unsafe_allow_html=
279         st.markdown('<div class="step-header"> 📍 Step 2: Points API E
280
281         with st.spinner("Fetching location data from National Weather
282             success, points_response = execute_curl_command(points_url
283
284         if success:
285             st.markdown('<div class="success-box"> ✅ Received {len(p
286
287             # Show a preview of the raw data
288             with st.expander("🔍 View Raw Points API Response (first !
289                 st.code(points_response[:500] + "... " if len(points_re
290         else:
291             st.markdown(f'<div class="error-box"> ❌ Failed to fetch p
292             st.stop()
293
294         st.markdown('</div>', unsafe_allow_html=True)
295
296     # Step 3: Extract Forecast URL
297     with st.container():
298         st.markdown('<div class="step-container">', unsafe_allow_html=
299         st.markdown('<div class="step-header"> 📍 Step 3: Extracting F
300
301         success, forecast_url = get_forecast_url_from_points_response(
302
303         if success:
304             st.markdown('<div class="success-box"> ✅ Forecast URL exti
305             st.code(forecast_url, language="text")
306         else:
307             st.markdown(f'<div class="error-box"> ❌ Failed to extract
308             st.stop()
309
310         st.markdown('</div>', unsafe_allow_html=True)
311
312     # Step 4: Forecast API Execution
313     with st.container():
314         st.markdown('<div class="step-container">', unsafe_allow_html=
315         st.markdown('<div class="step-header"> 📍 Step 4: Forecast API
316
317         with st.spinner("Fetching weather forecast data..."):
318             success, forecast_response = execute_curl_command(forecast
319
320         if success:
321             st.markdown(f'<div class="success-box"> ✅ Received {len(fi

```


 A green speech bubble with a white checkmark icon and the text "Copied!".

```

322
323         # Show a preview of the raw data
324         with st.expander("🔍 View Raw Forecast API Response (first 500 lines)"):
325             st.code(forecast_response[:500] + "...") if len(forecast_response) > 500
326         else:
327             st.markdown(f'<div class="error-box">❌ Failed to fetch forecast data')
328             st.stop()
329
330     st.markdown('</div>', unsafe_allow_html=True)
331
332     # Step 5: AI Processing
333     with st.container():
334         st.markdown('<div class="step-container">', unsafe_allow_html=True)
335         st.markdown('<div class="step-header">📅 Step 5: AI Processing')
336
337         with st.spinner("Claude is processing the weather data and creating a summary"):
338             success, summary = process_weather_response(forecast_response)
339
340         if success:
341             st.markdown('<div class="success-box">✅ Weather analysis complete')
342         else:
343             st.markdown(f'<div class="error-box">❌ Failed to process weather data')
344             st.stop()
345
346         st.markdown('</div>', unsafe_allow_html=True)
347
348     # Step 6: Results
349     st.markdown("---")
350     st.markdown("## 🌤️ Weather Forecast")
351     st.markdown(summary)
352
353     with col2:
354         # Real-time status updates
355         st.markdown("### 🔄 Process Status")
356
357         status_container = st.container()
358         with status_container:
359             st.markdown("""
360             <div class="info-box">
361             <strong>🔄 Agent Workflow:</strong><br>
362             ✅ Planning Phase<br>
363             ✅ Points API Call<br>
364             ✅ URL Extraction<br>
365             ✅ Forecast API Call<br>
366             ✅ Data Processing<br>
367             ✅ Results Generated
368             </div>
369             """, unsafe_allow_html=True)
370

```


 A purple speech bubble with a white checkmark icon and the text "Copied!".

```

371 st.markdown("### 🤖 What Makes This Agentic?")
372 st.markdown("""
373 - **🧠 Reasoning**: AI understands location formats
374 - **📋 Planning**: Generates appropriate API call sequences
375 - **🔧 Action**: Executes real-world API requests
376 - **📊 Processing**: Converts raw data to insights
377 - **🔄 Adaptation**: Handles different location types
378 """)
379
380 # Footer
381 st.markdown("---")
382 st.markdown("""
383 ### 📄 About This Demo
384
385 This application demonstrates **Agentic AI** principles using:
386 - **Amazon Bedrock** with Claude 4.5 Sonnet for intelligent reasoning
387 - **National Weather Service API** for real-time weather data
388 - **Streamlit** for interactive web interface
389
390 **⚠️ Important**: This uses official NWS data for educational purposes. For ci
391 """)
392
393 # Add some example queries
394 st.markdown("### 💡 Try These Examples:")
395 st.markdown("""
396 **Suggested locations to test:**
397 - **Seattle** - Major city (tests city name recognition)
398 - **90210** - ZIP code (tests postal code handling)
399 - **New York City** - Multi-word city (tests complex location parsing)
400 - **Miami, FL** - City with state (tests state abbreviations)
401 - **Chicago** - Another major city (tests different coordinates)
402 - **National park near Homestead in Florida** - Location description (tests AI
403 - **Largest City in California** - Descriptive query (tests knowledge-based lo
404
405 Simply copy any of these into the location input above and click "Get Weather
406 """)

```



Run Your Web Application

Now, in the terminal window, let's run the Streamlit web interface:

```
1 streamlit run weather_agent_web.py
```

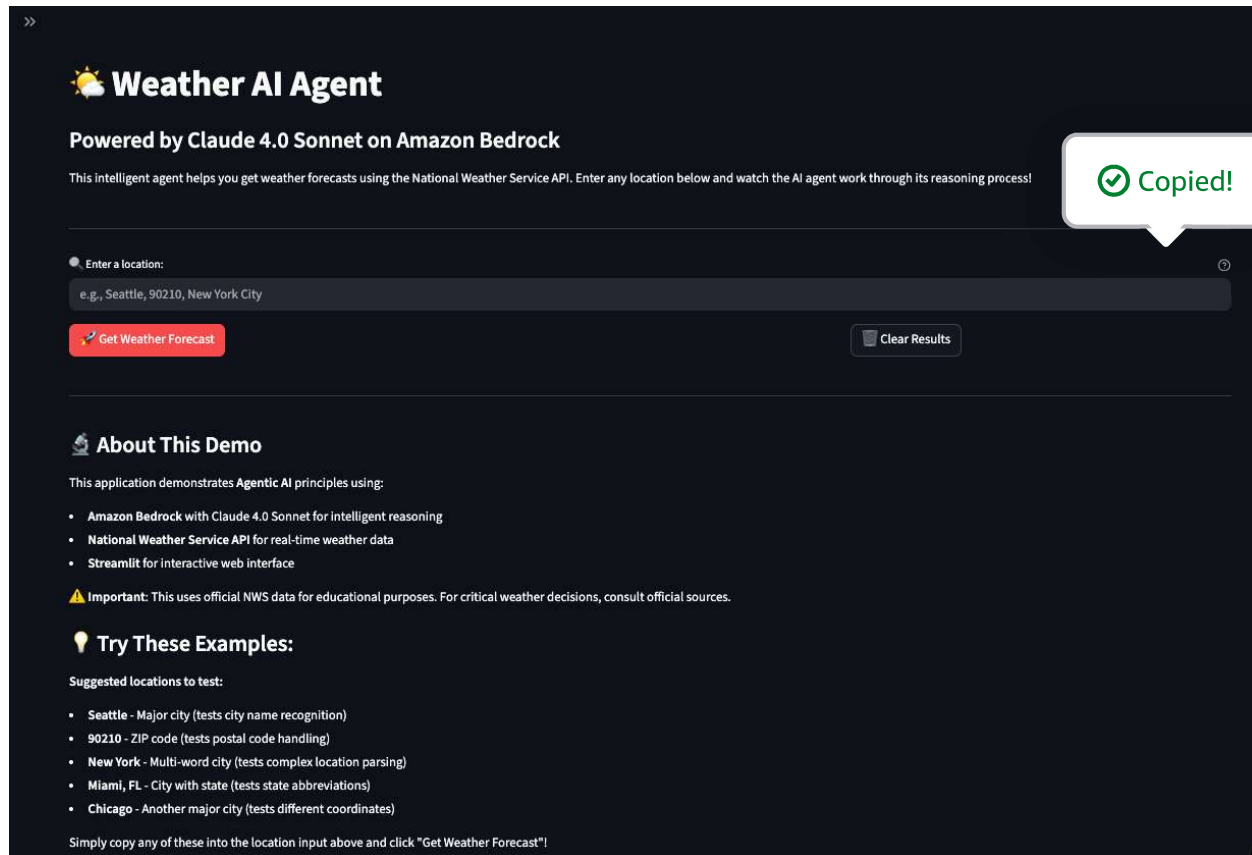


You should see output like:

You can now view your Streamlit app in your browser.

Local URL: `http://localhost:8501`

Network URL: `http://192.168.1.100:8501`



Explore the Web Interface

A few things worth doing while the app is running:

1. Try a mix of inputs — a major city, a ZIP code, and a vague description like "national park near Homestead in Florida". Watch the planning step pick different coordinates each time.
2. Expand the "View Raw API Response" sections under each step. The point of the analyst step is to turn those large JSON blobs into something you can read at a glance.
3. Watch the sidebar status update as each step completes.

Weather Analysis for: Seattle

Step 1: AI Planning Phase

Points API URL generated successfully!

https://api.weather.gov/points/47.6062,-122.3321

Step 2: Points API Execution

Received location data from NWS

View Raw Points API Response (first 500 characters)

Step 3: Extracting Forecast URL

Forecast URL extracted successfully!

https://api.weather.gov/points/52.125,68/forecast

Step 4: Forecast API Execution

Received 13,662 characters of forecast data

View Raw Forecast API Response (first 500 characters)

Step 5: AI Analysis Phase

Weather analysis complete!

Process Status

Agent Workflow:

- Planning Phase
- Points API Call
- URL Extraction
- Forecast API Call
- Data Processing
- Results Generated

What Makes This Agentic?

- Reasoning: AI understands location formats
- Planning: Generates appropriate API call sequences
- Action: Executes real-world API requests
- Processing: Converts raw data to insights
- Adaptation: Handles different location types

Weather Forecast

Seattle Weather Forecast

Location: Seattle, WA
Last Updated: September 8, 2025 at 2:26 AM PDT

Current Conditions & Today's Forecast

Overnight (Tonight): Partly cloudy skies with temperatures around 59°F. Light west-southwest winds at 2 mph with only a 12% chance of rain.

Monday (Today): Starting partly sunny, then becoming cloudier with a slight chance of rain showers after 5 PM. High temperature reaching 74°F with light winds from the west-northwest at 1-5 mph. Rain chance increases to 20% by evening.

3-Day Outlook

Tuesday, September 9

- High: 70°F | Low: 59°F
- Conditions: Chance of rain showers throughout the day
- Rain Probability: 50-52%
- Winds: Light north winds 1-5 mph
- Rainfall: Less than 0.1 inches expected

Wednesday, September 10

- High: 71°F | Low: 59°F
- Conditions: Partly sunny with decreasing shower chances
- Rain Probability: 30% during the day, 20% at night
- Winds: Light northwest winds around 3 mph
- Trend: Weather beginning to improve

Thursday, September 11

- High: 72°F | Low: 59°F
- Conditions: Partly sunny with only slight shower chances
- Rain Probability: 20-24%
- Winds: Light south-southwest winds around 5 mph

What to Observe

The planning step is where the agent feels intelligent. It recognises "Seattle" as a city, "90210" as a ZIP code, and "largest city in California" as a knowledge query, and emits a different Points API URL for each.

The analysis step is where the agent feels useful. The forecast JSON is hundreds of lines covering a week of three-hour periods; Claude condenses it into a few sentences a human actually wants to read.

To see the failure modes, try a misspelled city, a location outside the US (the NWS only covers US territory), or a nonsense input. The error handling in the four functions surfaces useful messages rather than crashing the app.

CLI vs. Web

Both versions call the same four functions and follow the same six-step workflow. The CLI version is better when you want to script the agent or pipe its output somewhere. The web version is better for demos and for users who would rather not look at a terminal. The agent logic does not change between them — only the presentation does.

What's Next

You have a working agent in two interfaces. The next section steps back from the code to look at the patterns that made it work, and how you might apply them to your own problems.

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

[Privacy policy](#)

[Terms of use](#)

[Cookie preferences](#)